Limitations. In all of the experiments reported in this manuscript, we show that RUDDER significantly outperforms other methods for delayed reward problems. However, RUDDER might not be effective when the reward is not delayed since LSTM learning takes extra time and has problems with very long sequences. Furthermore, reward redistribution may introduce disturbing spurious reward signals.

4 Experiments

RUDDER is evaluated on three artificial tasks with delayed rewards. These tasks are designed to show problems of TD, MC, and potential-based reward shaping. RUDDER overcomes these problems. Next, we demonstrate that RUDDER also works for more complex tasks with delayed rewards. Therefore, we compare RUDDER with a Proximal Policy Optimization (PPO) baseline on 52 Atari games. All experiments use finite time horizon or absorbing states MDPs with $\gamma = 1$ and reward at episode end. For more information see Section A4.1 in the appendix.

Artificial Tasks (I)–(III). Task (I) shows that TD methods have problems with vanishing information for delayed rewards. Goal is to learn that a delayed reward is larger than a distracting immediate reward. Therefore, the correct expected future reward must be assigned to many state-action pairs. Task (II) is a variation of the introductory pocket watch example with delayed rewards. It shows that MC methods have problems with the high variance of future unrelated rewards. The expected future reward that is caused by the first action has to be estimated. Large future rewards that are not associated with the first action impede MC estimations. Task (III) shows that potential-based reward shaping methods have problems with delayed rewards. For this task, only the first two actions are relevant, to which the delayed reward has to be propagated back.

The tasks have different delays, are tabular (Q-table), and use an ϵ-greedy policy with $\epsilon = 0.2$. We compare RUDDER, MC, and TD(λ) on all tasks, and Monte Carlo Tree Search (MCTS) on task (I). Additionally, on task (III), SARSA(λ) and reward shaping are compared. We use $\lambda = 0.9$ as suggested [128]. Reward shaping methods are the original method, look-forward advice, and look-back advice with three different potential functions. RUDDER uses an LSTM without output and forget gates, no lessons buffer, and no safe exploration. For all tasks contribution analysis is performed with difference of return predictions. A Q-table is learned by an exponential moving average of the redistributed reward (RUDDER's Q-value estimation) or by Q-learning. Performance is measured by the learning time to achieve 90% of the maximal expected return. A Wilcoxon signed-rank test determines the significance of performance differences between RUDDER and other methods.

(I) Grid World shows problems of TD methods with delayed rewards. The task illustrates a time bomb that explodes at episode end. The agent has to defuse the bomb and then run away as far as possible since defusing fails with a certain probability. Alternatively, the agent can immediately run away, which, however, leads to less reward on average. The Grid World is a 31×31 grid with *bomb* at coordinate $[30, 15]$ and *start* at $[30 - d, 15]$, where d is the delay of the task. The agent can move *up*, *down*, *left*, and *right* as long as it stays on the grid. At the end of the episode, after $\lfloor 1.5d \rfloor$ steps, the agent receives a reward of 1000 with probability of 0.5, if it has visited *bomb*. At each time step, the agent receives an immediate reward of $c \cdot t \cdot h$, where c depends on the chosen action, t is the current time step, and h is the Hamming distance to *bomb*. Each move toward the *bomb*, is immediately penalized with $c = -0.09$. Each move away from the *bomb*, is immediately rewarded with $c = 0.1$. The agent must learn the Q-values precisely to recognize that directly running away is not optimal. Figure 1(I) shows the learning times to solve the task vs. the delay of the reward averaged over 100 trials. For all delays, RUDDER is significantly faster than all other methods with p-values $< 10^{-12}$. Speed-ups vs. MC and MCTS, suggest to be exponential with delay time. RUDDER is exponentially faster with increasing delay than $Q(\lambda)$, supporting Theorem A8 in the appendix. **RUDDER significantly outperforms all other methods.**

(II) The Choice shows problems of MC methods with delayed rewards. This task has probabilistic state transitions, which can be represented as a tree with states as nodes. The agent traverses the tree from the root (initial state) to the leafs (final states). At the root, the agent has to choose between the left and the right subtree, where one subtree has a higher expected reward. Thereafter, it traverses the tree randomly according to the transition probabilities. Each visited node adds its fixed share to the final reward. The delayed reward is given as accumulated shares at a leaf. The task is solved when

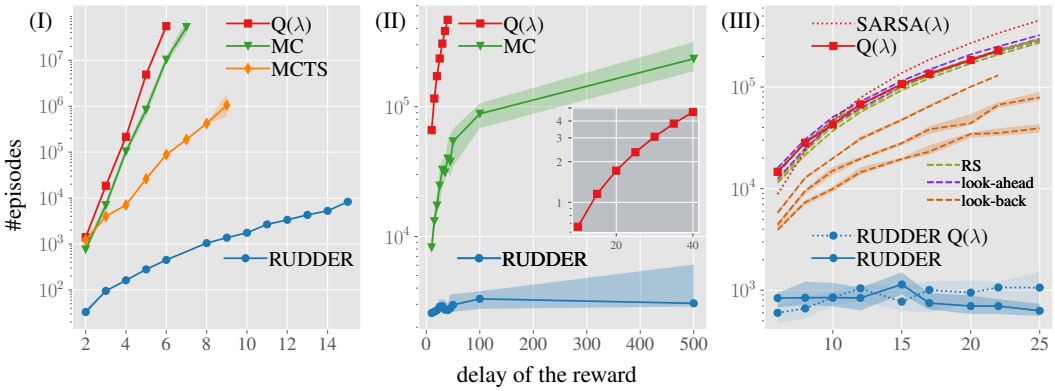

Figure 1: Comparison of RUDDER and other methods on artificial tasks with respect to the learning time in episodes (median of 100 trials) vs. the delay of the reward. The shadow bands indicate the 40% and 60% quantiles. In (II), the y-axis of the inlet is scaled by 10^5. In (III), reward shaping (RS), look-ahead advice (look-ahead), and look-back advice (look-back) use three different potential functions. In (III), the dashed blue line represents RUDDER with $Q(\lambda)$, in contrast to RUDDER with Q-estimation. In all tasks, RUDDER significantly outperforms all other methods.

the agent always chooses the subtree with higher expected reward. Figure 1(II) shows the learning times to solve the task vs. the delay of the reward averaged over 100 trials. For all delays, RUDDER is significantly faster than all other methods with p-values $< 10^{-8}$. The speed-up vs. MC, suggests to be exponential with delay time. RUDDER is exponentially faster with increasing delay than $Q(\lambda)$, supporting Theorem A8 in the appendix. **RUDDER significantly outperforms all other methods.**

(III) Trace-Back shows problems of potential-based reward shaping methods with delayed rewards. We investigate how fast information about delayed rewards is propagated back by RUDDER, $Q(\lambda)$, SARSA(λ), and potential-based reward shaping. MC is skipped since it does not transfer back information. The agent can move in a 15×15 grid to the 4 adjacent positions as long as it remains on the grid. Starting at $(7, 7)$, the number of moves per episode is $T = 20$. The optimal policy moves the agent up in $t = 1$ and right in $t = 2$, which gives immediate reward of -50 at $t = 2$, and a delayed reward of 150 at the end $t = 20 = T$. Therefore, the optimal return is 100. For any other policy, the agent receives only an immediate reward of 50 at $t = 2$. For $t \leqslant 2$, state transitions are deterministic, while for $t > 2$ they are uniformly distributed and independent of the actions. Thus, the return does not depend on actions at $t > 2$. We compare RUDDER, original reward shaping, look-ahead advice, and look-back advice. As suggested by the authors, we use SARSA instead of Q-learning for look-back advice. We use three different potential functions for reward shaping, which are all based on the reward redistribution (see appendix). At $t = 2$, there is a distraction since the immediate reward is -50 for the optimal and 50 for other actions. RUDDER is significantly faster than all other methods with p-values $< 10^{-17}$. Figure 1(III) shows the learning times averaged over 100 trials. **RUDDER is exponentially faster than all other methods and significantly outperforms them.**

Atari Games. RUDDER is evaluated with respect to its learning time and achieves scores on Atari games of the Arcade Learning Environment (ALE) [11] and OpenAI Gym [18]. RUDDER is used on top of the TRPO-based [113] policy gradient method PPO that uses GAE [114]. Our PPO baseline differs from the original PPO baseline [115] in two aspects. (i) Instead of using the sign function of the rewards, rewards are scaled by their current maximum. In this way, the ratio between different rewards remains unchanged and the advantage of large delayed rewards can be recognized. (ii) The safe exploration strategy of RUDDER is used. The entropy coefficient is replaced by Proportional Control [16, 12]. A coarse hyperparameter optimization is performed for the PPO baseline. For all 52 Atari games, RUDDER uses the same architectures, losses, and hyperparameters, which were optimized for the baseline. The only difference to the PPO baseline is that the policy network predicts the value function of the redistributed reward to integrate reward redistribution into the PPO framework. Contribution analysis uses an LSTM with differences of return predictions. Here Δ is the pixel-wise difference of two consecutive frames augmented with the current frame. LSTM training and reward redistribution are restricted to sequence chunks of 500 frames. Source code is provided upon publication.

	RUDDER	baseline	delay	delay-event
Bowling	192	56	200	strike pins
Solaris	1,827	616	122	navigate map
Venture	1,350	820	150	find treasure
Seaquest	4,770	1,616	272	collect divers

Table 1: Average scores over 3 random seeds with 10 trials each for delayed reward Atari games. "delay": frames between reward and first related action. RUDDER considerably improves the PPO baseline on delayed reward games.

Policies are trained with no-op starting condition for 200M game frames using every 4th frame. Training episodes end with losing a life or at maximal 108K frames. All scores are averaged over 3 different random seeds for network and ALE initialization. We asses the performance by the learning time and the achieved scores. First, we compare RUDDER to the baseline by average scores per game throughout training, to assess learning speed [115]. For 32 (20) games RUDDER (baseline) learns on average faster. Next, we compare the average scores of the last 10 training games. For 29 (23) games RUDDER (baseline) has higher average scores. In the majority of games RUDDER, improves the scores of the PPO baseline. To compare RUDDER and the baseline on Atari games that are characterize by delayed rewards, we selected the games Bowling, Solaris, Venture, and Seaquest. In these games, high scores are achieved by learning the delayed reward, while learning the immediate reward and extensive exploration (like for Montezuma's revenge) is less important. The results are presented in Table 1. For more details and further results see Section A4.2 in the appendix. Figure 2 displays how RUDDER redistributes rewards to key events in Bowling. **At delayed reward Atari games, RUDDER considerably increases the scores compared to the PPO baseline.**

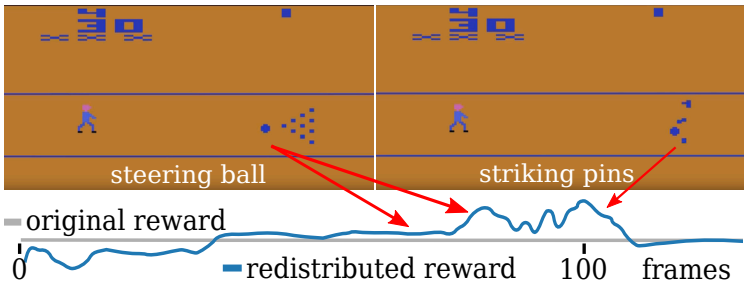

Figure 2: RUDDER redistributes rewards to key events in the Atari game Bowling. Originally, rewards are delayed and only given at episode end. The first 120 out of 200 frames of the episode are shown. RUDDER identifies key actions that steer the ball to hit all pins.

Conclusion. We have introduced RUDDER, a novel reinforcement learning algorithm based on the new concepts of reward redistribution and return decomposition. On artificial tasks, RUDDER significantly outperforms TD(λ), MC, MCTS and reward shaping methods, while on Atari games it improves a PPO baseline on average but most prominently on long delayed rewards games.

Acknowledgments

This work was supported by NVIDIA Corporation, Merck KGaA, Audi.JKU Deep Learning Center, Audi Electronic Venture GmbH, Janssen Pharmaceutica (madeSMART), TGW Logistics Group, ZF Friedrichshafen AG, UCB S.A., FFG grant 871302, LIT grant DeepToxGen and AI-SNN, and FWF grant P 28660-N31.

References

References are provided in Section A11 in the appendix.